# 기본 자료형과 문자열형

# 학습목표

- 기본 자료형의 종류와 특징을 이해하고 각 자료형의 용도에 맞는 데이터를 처리할 수 있다.
- 문자열(String) 자료형의 기본적인 개념을 이해하고, 문자열 관련 연산 및 함수(문자열 연결, 슬라이싱, 길이 확인 등)를 활용할 수 있다.

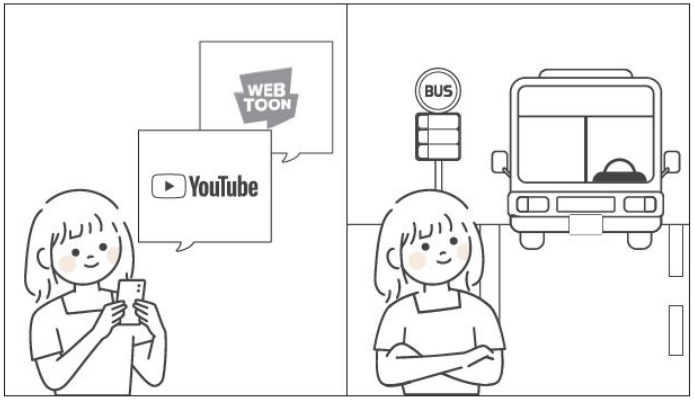
# 목차

- 변수 되돌아 보기
- 기본 자료형
- 문자열형

# 4.1 변수 되돌아 보기

- 변수란 값을 저장할 수 있는 기억 공간
- 어떤 값에 대한 이름표

- 보라가 ( 웹툰 )을 본다.
- 보라가 ( 유튜브 ) 를 본다.
- 보라가 (스마트폰)을 본다.
- 보라가 (버스)를 본다.




보라가 (  )을/를 봅니다

계속해서 변하는 데이터가 들어오는 부분()을 변수로

지정하여 규칙을 만들면 좀 더 유연하고 효율적

# 4.1 변수 되돌아 보기

- 변수의 예 : 연락처 어플

| 변수 만들기 |
| --- |
| 연락처 저장에 필요한 데이터를 저장할 변수는? |

↓

| 변수에 사용자가 입력한 자료 저장하기 |
| --- |
| 각 변수에 사용자가 입력한 데이터를 저장 |

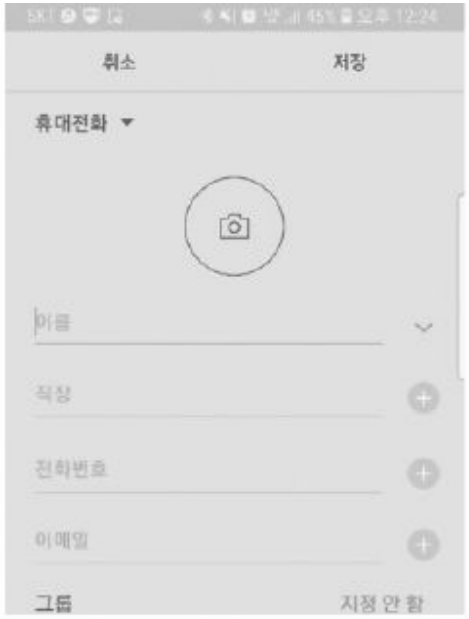



그림 4.2

```
name=input('이름:')
job=input('직장:')
tel=input('전화번호:')
email=input('이메일:')
```

# 4.2 자료형

- 자료 : 컴퓨터가 처리할 수 있는 숫자나 문자 등을 말함
- 자료형 : 자료의 특성에 따라 자료형을 구분
- 자료의 종류 구분 이유
  - 컴퓨터에게 어떻게 연산을 하는지 알려주기 위해서 '**변수는 정수형이야', 혹은 '**변수는 문자야'라는 의미로 자료의 타입을 구분함

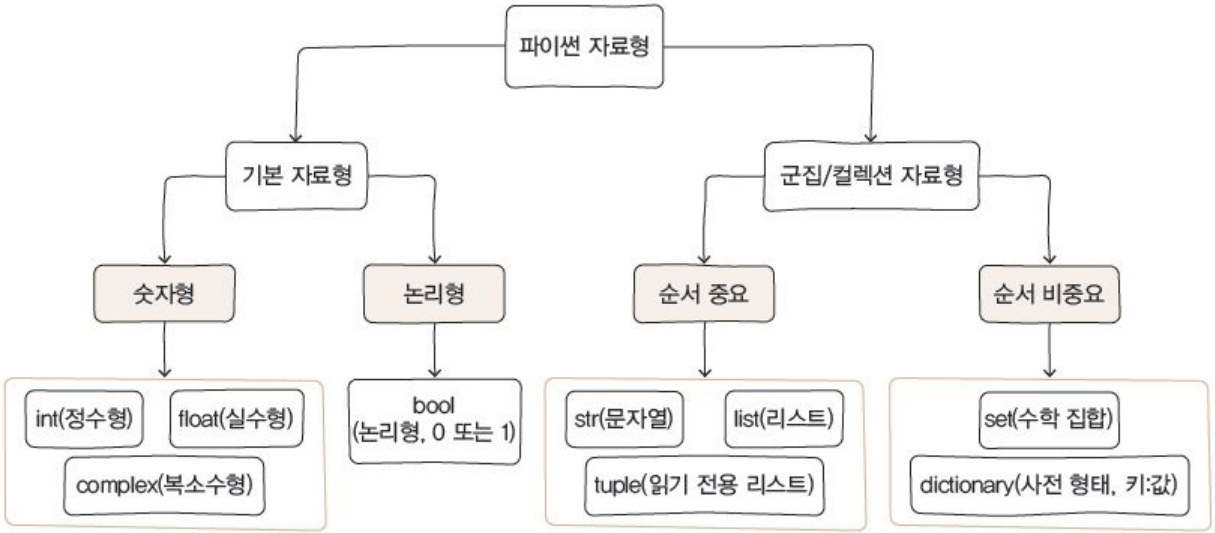



그림 4.3

# 4.2 자료형

파이썬은 변수의 타입을 선언하지 않아도 값이 할당되면 자동으로 변수의 자료형이 결정된다

표 4.1

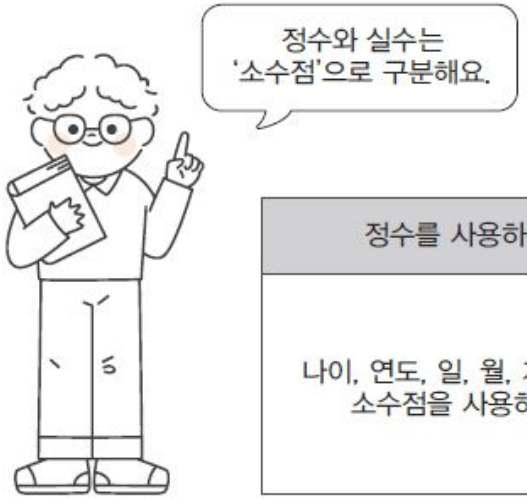
| 코드 | 실행 결과 | 데이터형 |
|---|---|---|
| a = 5<br>type(a) | int | 정수형 |
| a = 5.3<br>type(a) | float | 실수형 |
| a = 'python'<br>type(a) | str | 문자열형 |
| a = True<br>type(a) | bool | 부울형 |
| a=[1,2,3,3]<br>type(a) | list | 리스트형 |
| a={1,2,3,3}<br>type(a) | set | 세트형 |
| a=(1,2,3,3)<br>type(a) | tuple | 튜플형 |
| a={1: 'python', 2: 'Java'}<br>type(a) | dict | 딕셔너리형 |

# 4.2 자료형

## (1) 숫자 자료형

– '정수'와 '실수'로 구분

| | |
|---|---|
| 3 | int (정수형) |
| 3.0 | float형 (실수형) |




| 정수를 사용하는 경우 | 실수를 사용하는 경우 |
|---|---|
| 나이, 연도, 일, 월, 개수, 등수 등<br>소수점을 사용하지 않음 | 길이, 무게, 과목평균, 온도, 부피 등<br>소수점을 사용함 |

그림 4.4

# 4.2 자료형

- input함수를 사용하면 사용자가 입력한 값이 string으로 인식된다. type함수를 사용하면 자료 형을 확인할 수 있다.

예제 4.1 정수형과 문자열형

```
num=input('수를 입력하세요')
print(type(num))
```

실행결과

```
수를 입력하세요 45 Enter
<class 'str'>
```

# 4.2 자료형

- 입력 받은 값을 숫자형식으로 변경하려면 원하는 값 형태로 강제 형 변환을 한다.
- 정수형으로 변환은 int()를 사용한다

예제 4.2 정수형 형변환

```
num1=int(input('1st:'))
num2=int(input('2st:'))
print(num1+num2)
```

실행결과

```
1st:3 Enter
2st:3 Enter
6
```

# 4.2 자료형

- 실수형으로 입력 받기 위해서는 float()를 사용한다.

예제 4.3 실수형 형변환

```
num1=float(input('1st:'))
num2=float(input('2st:'))
print(num1+num2)
```

실행결과

```
1st:3 Enter
2st:3 Enter
6.0
```

# 4.2 자료형

- 정수와 정수의 나눗셈 연산

예제 4.4 나눗셈 연산 결과

```
>>> 4/2
2.0
```

```
>>> 4/2
2.0
```

! 파이썬에서는 5/2와 같은 정수형의 나눗셈의 결과를 실수형 2.5로 정확하게 나타내기 위해 나눗셈의 연산의 경우 데이터형과 상관없이 모두 float로 할당

# 4.2 기본 자료형

- 정수와 실수형의 연산

예제 4.5 정수형과 실수형 연산

```
result = 3 * 3.0
print(result)  # 출력: 9.0
print(type(result))  # 출력: <class
'float'>
```

```
result = 3 + 3.0
print(result)  # 출력: 6.0
print(type(result))  # 출력: <class
'float'>
```

# 4.2 기본 자료형

## (2) 문자열형

- 문자 그대로를 이용
- 파이썬에서는 문자를 따옴표인 Single Quotation(‘) 또는 Double Quotation(“”)를 이용
- 사용자로부터 입력 받은 값을 사용한다면, 입력함수만 사용

예제 4.6 입력 함수와 문자형

```
id=input('아이디: ')
pwd=input('비밀번호: ')
print(type(id), type(pwd))
```

```
아이디: python Enter
비밀번호: 1234 Enter
<class 'str'> <class 'str'>
```

# 4.2 자료형

## ① 문자형 병합

- 두 수를 더하는 연산자는 '+'이다.
- 이 연산자를 문자형 사이에 사용하여 문자형 데이터들을 합친다.

예제 4.7 문자형의 병합

```
name='동동'+'일'
print(name)
```

실행결과

```
동동일
```

```
s='@'*5
print(s)
```

실행결과

```
@@@@@
```

# 4.2 자료형

## ② 문자열의 길이

- 문자열의 경우 총 몇 개의 문자가 있는지 파악 할 때 사용
- len()은 문자 또는 문자열의 길이를 알려준다.

**예제 4.8** len() 사용법

```
id=input("아이디:")
print(len(id))
```

```
아이디:python Enter
6
```

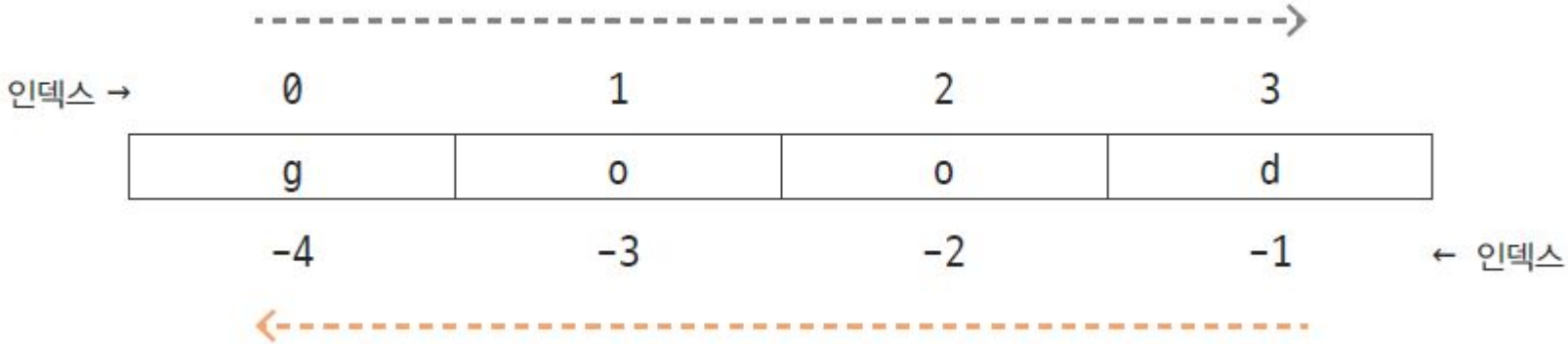
**예제 4.9** 공백 포함된 문자열 길이

```
x="I am Happy"
print(len(x))
```

# 4.2 자료형

## ③ 문자열의 인덱싱

- 인덱스 : 순서가 있는 자료형에서 각 요소의 순서(위치)
  - 앞에서부터 순서를 매길 때는 처음 문자의 인덱스는 0이고 그 다음 문자들의 인덱스는 1씩 증가함
  - 뒤에서부터 순서를 매길때는 마지막 문자가 인덱스 -1로 시작하여 1씩 감소함



# 4.2 자료형

## ③ 문자열의 인덱싱

- 인덱싱 : 인덱스를 사용하여 문자에 접근하는 것을 의미함

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| g | o | o | d | | l | u | c | k |

그림 4.6

예제 4.11 문자열 인덱싱

```
x='good luck'
x[5]
```

실행결과

```
1
```

# 4.2 자료형

## ④ 문자열 슬라이싱

- 슬라이싱: 인덱스를 활용하여 문자열의 일부분을 추출하는 것

| 슬라이싱 형식 | 예 |
|---|---|
| 양수 인덱스를 사용할 때 형식:<br>문자열[시작 인덱스:끝 인덱스+1:간격] | `s = 'abcdef'`<br>`print(s[1:4])  #'bcd'`<br>`print(s[-4:-1]) #'cde'` |
| 음수 인덱스를 사용할 때 형식:<br>문자열[시작 인덱스:끝 인덱스-1:간격] | |

- 문자열을 일부분 추출할 때 사용하는 범위

[n:m] : n번째부터 m-1번째까지를 의미
[:m] : 처음부터 m-1번째까지를 의미
[n:] : n부터 문자열의 마지막까지를 의미

# 4.2 자료형

## ④ 문자열 슬라이싱

예제 4.12 문자열과 인덱스

```
words="good luck"
print(words[5:9])
print(words[5:])
print(words[:4])
print(words[::2])
print(words[::-1])
```

실행결과

```
luck
luck
good
go uk
kcul doog
```

# 4.3 자료형

④ 문자열 조작함수

| 함수 | 기능 |
|---|---|
| 문자열.replace(x1, x2) | 문자열에서 x1을 x2로 변경 |
| 문자열.split(x) | 문자열을 x 기준으로 분리(※ 공백을 기준으로 분리하는 것은 default이므로 split()으로 작성) |
| 문자열.upper() | 문자열을 대문자로 변환 |
| 문자열.lower() | 문자열을 소문자로 변환 |
| 문자열.find(x) | 문자열에서 x의 인덱스 찾기 |
| 문자열.count(x) | 문자열에서 x의 개수 |
| 문자열.isdigi() | 문자열이 숫자로만 구성되어 있는지 판별 |
| 문자열.isalpha() | 문자열이 문자로만 구성되었는지 판별 |
| 문자열.isalnum() | 문자열이 문자 또는 숫자로만 구성되었는지 판별(예: 'A12 as'의 경우 공백(특수문자)이 포함되어 있어서 False를 출력함) |

# 4.3 자료형

예제 4.13 문자열 함수 사용 예

```
s = '간절히 바라는 마음이 있어야 Magic이 일어날 수 있어'
# 원본 문장 출력
print('원본 문장:', s)
# 1. 문자열.replace(x1, x2): '공백'을 '-'로 변경
print('1:', s.replace(' ', '-'))
# 2. 문자열.split(x): 단어들을 공백 기준으로 분리
print('2:', s.split())
# 3. 문자열.upper(): 대문자로 변환
print('3:', s.upper())
# 4. 문자열.lower(): 소문자로 변환
```

```
print('4:', s.lower())
# 5. 문자열.find(x): '바'의 인덱스 찾기
print('5:', s.find('바'))
# 6. 문자열.count(x): '이' 문자의 개수 세기
print('6:', s.count('이'))
# 7. 문자열.isdigit(): 숫자로만 이루어졌는지 판별
print('7:', s.isdigit())
# 8. 문자열.isalpha(): 문자로만 이루어졌는지 판별
print('8:',s.isalpha())
# 9. 문자열.isalnum(): 문자 또는 숫자로만 이루어졌는지 판별
print('9:', s.isalnum())
```

# 수고하셨습니다.